

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ федерального государственного бюджетного
образовательного учреждения высшего образования
«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ
Г.В.ПЛЕХАНОВА»
Техникум Пермского института (филиала)

Практическая работа №3
по дисциплине «Поддержка и тестирование программных модулей»
Тема: «Разработка приложения управления складами»
Студента группы ИПо-32
Новикова Александра Олеговна

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1. Стек технологий	4
2. Архитектура	4
3. Реализация бизнес-модулей программы	4
3.1 Модуль организаций	5
3.2 Модуль складов	5
3.3 Модуль товаров	5
3.4 Модуль накладных	6
ЗАКЛЮЧЕНИЕ	8

ВВЕДЕНИЕ

В данной работе представлено техническое описание информационной системы складского учета, разработанной с применением современных средств платформы .NET и архитектурных паттернов, ориентированных на сопровождаемость и масштабируемость. В качестве базовой платформы выбран .NET 8.0 LTS, пользовательский интерфейс реализован на базе WPF с использованием шаблона Model-View-ViewModel. Архитектура системы построена по принципу четкого разделения ответственности: бизнес-логика и модели данных инкапсулированы в отдельной библиотеке классов, а взаимодействие с пользователем вынесено в слой представлений с использованием контейнера зависимостей и единой системы команд.

Документ структурирован следующим образом: сначала описывается общий стек технологий и архитектурные решения, лежащие в основе проекта, после чего детально рассматриваются ключевые модули системы (организации, склады, товары). Для каждого модуля приводится структура данных, описание сервисного слоя, реализующего бизнес-правила, а также особенности пользовательского интерфейса и механизмы взаимодействия с данными. Материал предназначен для разработчиков, аналитиков и сопровождающих лиц, работающих над развитием проекта.

1. Стек технологий

Платформа - .NET 8.0 (с долгосрочной поддержкой)

Бэкенд - библиотека классов .NET 8.0

Фронтэнд (UI) - WPF .NET 8.0

2. Архитектура

Система придерживается архитектурного шаблона Model-View-ViewModel.

Данные в системе хранятся и используются в виде объектов классов Models, структура объектов задана в соответствии с предметной областью информационной системы.

Хранение данных организовано в вышеупомянутых объектах, данные сгруппированы в коллекции и связаны иерархией: Организации -> Склады (1:M); Склады -> Товары (1:M); Склады -> Накладные (1:M), Накладные->Товары (1:M)

Манипуляции данными осуществляются на уровне объектов классов Services - в них реализована бизнес-логика работы с объектами предметной области.

Все коллекции для хранения данных и некоторые сервисы создаются в контейнере ApplicationContext в начале работы программы

UI приложения состоит из нескольких видов компонентов: отдельные окна (основные, модальные), пользовательские контроллеры (используются внутри окон)

Каждый компонент представляет из себя разделенные представление страницы(файл разметки *.xaml) и логику обработки событий на странице (файл *.cs)

Компоненты на страницах (кнопки) связаны с методами их обработки при помощи статических объектов-команд RoutedCommand, заданных в отдельном объекте WindowsCommands.

3. Реализация бизнес-модулей программы

Часть программы, отвечающая за бизнес-логику и хранение данных реализована в .Net библиотеке WarehouseData (Contexts, Models, Services). Часть, отвечающая за пользовательский интерфейс реализована в WPF .Net (Views, ModalWindows, Controls)

3.1 Модуль организаций

Организационная единица хранится в программе в виде класса `organization`. Ключевым полем для него является название организации, которое не может быть пустым. Объект организации идентифицируется по отдельному индексному полю в своей структуре. Также в нем хранится коллекция `Warehouses` - складов, принадлежащих организации.

Логика работы с организацией (добавление, изменение, удаление) реализована в классе `ServiceOrgs`

Для организации используется представление `OrgsView`. В нем выводится список организаций, который хранится в виде коллекции в контейнере `ApplicationContext`. Операции CRUD применяются над организацией через контекстное меню над списком и через ПКМ. При их нажатии вызывается модальное окно `OrgsModWindow??` для манипуляции с данными организации.

Выбрав организацию можно перейти к окну складов и товаров

3.2 Модуль складов

Склад хранится в программе в виде класса `Warehouse`. Поле названия склада не должно быть пустым. Также должен быть обязательный индекс организации. Объект склада идентифицируется по отдельному индексному полю в своей структуре. Также в нем хранится коллекция `Products` - товаров, принадлежащих складу.

Логика работы со складом (добавление, изменение, удаление) реализована в классе `ServiceWhs`.

Для представления складов в пользовательском интерфейсе используется класс `MainWindow` и `UserControlWarehouse`. `MainWindow` это отдельное окно с контекстным меню, через которое можно взаимодействовать с объектами складов (CRUD). `UserControlWarehouse` - пользовательский элемент, который встроен в `MainWindow`. В левом столбце элемента расположен список складов выбранной ранее организации. С объектом склада можно взаимодействовать, выделив его и используя в контекстном меню, или нажав ПКМ на него. Для манипуляции с данными склада создано модальное окно `WinModWarehouse??`

3.3 Модуль товаров

Товар хранится в программе в виде класса `Product`. Для товара есть поле артикль, описательные поля, поле кодичества. У товара есть 3 составных поля, которые являются элементами пользовательских классов, также заданных в системе:

Категория - класс Category,
Производитель - класс Manufacturer,
Поставщик - класс Supplier.

Также у товара есть идентификатор склада, на котором он хранится. Логика работы со складом (бизнес-валидация полей, добавление, изменение, удаление) реализована в классе ServiceProds. Для представления товаров в пользовательском интерфейсе используются классы MainWindow, UserControlWarehouse, UserControlOrders. MainWindow дает возможность взаимодействовать с товаром через контекстное меню.

UserControlWarehouse в правой части содержит таблицу, в которой показываются товары, принадлежащие одному складу. Вывести товары, принадлежащие складу можно через контекстное меню или два раза нажав ЛКМ на склад слева. В данном контексте список товаров предназначен только для просмотра на складах, поэтому в таблице с ними взаимодействовать нельзя.

Товары можно создавать, изменять, удалять в накладных, в UserControlOrders. UserControlOrders имеет внешний вид и функции, аналогичные UserControlWarehouse. Слева в столбце расположены Накладные, принадлежащие складу. Справа - таблица с товарами, принадлежащими выделенной накладной. Манипулировать с товарами (создавать, изменять, удалять) можно при помощи контекстного меню (Товар) и также по нажатию ПКМ в таблице.

3.4 Модуль накладных

Накладная хранится в виде класса Order с основными полями: номер накладной, дата, тип (входящая, исходящая), статус (утверждена, не утверждена). Также хранится идентификатор склада, к которому относится накладная и коллекция товаров, которые должны поступить или списаться со склада.

Логика работы с накладной реализована в сервисе ServiceOrds. Накладную можно добавить, удалить (привязать или отвязать от склада). Также накладную можно утвердить (поменять статус). При создании накладной она сохраняется в статусе не утверждена. В таком состоянии к ней можно добавлять/удалять товары, менять их. В процессе утверждения накладная передает свои товары в объект склада или списывает, в зависимости от типа (входящий, исходящий). После подтверждения накладную нельзя удалить, нельзя добавить в нее отвал. Она остается привязана к складу.

Для представления модуля накладных используются классы `MainWindow`, `UserControlOrders`. С помощью контекстного меню `MainWindow` можно переключаться между `UserControl` складами и накладными (Склады->Список складов, Накладные->Открыть список накладных). При переходе к накладным, появляется список накладных, принадлежащих выделенному классу.

ЗАКЛЮЧЕНИЕ

Описанная архитектура и выбранный технологический стек формируют надежный фундамент для разработки и дальнейшей поддержки системы складского учета. Разделение кода на библиотеку бизнес-логики (WarehouseData) и WPF-слой интерфейса, применение MVVM, централизованное создание зависимостей в ApplicationContext и унифицированная обработка действий через RoutedCommand обеспечивают высокую модульность, упрощают модульное тестирование и снижают риски при внесении изменений. Реализованные модули покрывают базовый жизненный цикл работы с предметной областью, поддерживая иерархическую связь сущностей и полный набор CRUD-операций с удобным визуальным взаимодействием.

В перспективе система может быть расширена за счет подключения внешней СУБД для персистентного хранения данных, внедрения механизмов авторизации и ролевой модели, а также добавления функций экспорта, импорта и пакетной обработки данных. Представленное техническое описание служит исчерпывающей базой для понимания внутренней организации проекта, облегчает онбординг новых разработчиков и задаёт четкие стандарты для дальнейшего развития функционала.